

VR.net: A Real-world Dataset for Virtual Reality Motion Sickness Research

Elliott Wen*
The University of Auckland

Chitralekha Gupta
National University of Singapore

Prasanth Sasikumar
National University of Singapore

Mark Billinghurst†
The University of Auckland

James Wilmott
Meta Reality Labs

Emily Skow
Meta Reality Labs

Arindam Dey
Meta Reality Labs

Suranga Nanayakkara‡
National University of Singapore

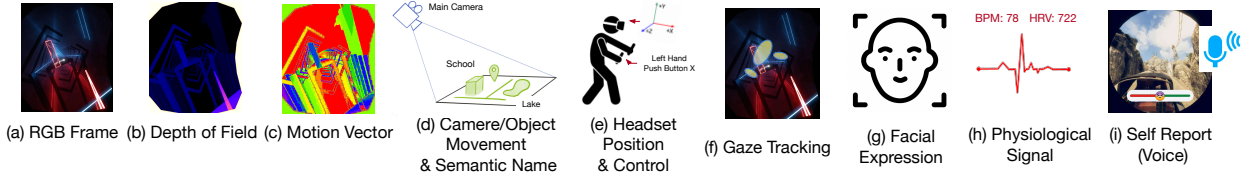


Fig. 1: VR.net encompasses 165-hour gameplay videos of 100 real-world VR games, representing 10 representative genres, and involving the participation of 500 individuals. For each video frame, VR.net assigns 24 types of labels that are known to induce motion sickness.

Abstract—Researchers have used machine learning approaches to identify motion sickness in VR experience. These approaches would certainly benefit from an accurately labeled, real-world, diverse dataset that enables the development of generalizable ML models. We introduce ‘VR.net’, a dataset comprising 165-hour gameplay videos from 100 real-world games spanning ten diverse genres, evaluated by 500 participants. VR.net accurately assigns 24 motion sickness-related labels for each video frame, such as camera/object movement, depth of field, and motion flow. Building such a dataset is challenging since manual labeling would require an infeasible amount of time. Instead, we implement a tool to automatically and precisely extract ground truth data from 3D engines’ rendering pipelines without accessing VR games’ source code. We illustrate the utility of VR.net through several applications, such as risk factor detection and sickness level prediction. We believe that the scale, accuracy, and diversity of VR.net can offer unparalleled opportunities for VR motion sickness research and beyond. We also provide access to our data collection tool, enabling researchers to contribute to the expansion of VR.net.

1 INTRODUCTION

VR gaming has gained widespread popularity in recent years, with the annual market revenue projected to reach \$87 billion by 2030¹. However, up to 40% of users suffer from VR motion sickness with symptoms like fatigue, disorientation, and nausea [1]. These adverse effects can severely undermine the user experience.

To date, considerable research [2–4] has uncovered that VR content is the most influential factor for VR sickness. Therefore, many VR game stores desire to identify motion sickness risk factors and predict comfort levels in VR content. The results could inform users about potentially unpleasant feelings. They could also assist game developers in adjusting the game-level design for a safer experience. Initially, this evaluation process was conducted by human experts, which quickly failed to cope with the growing number of VR games. Recently, researchers have proposed Machine Learning (ML) approaches to estimate cybersickness directly from the content [4, 5]. Despite progress, many of these studies pointed out that the limited training datasets constrain their models. To improve accuracy, they would require a large-scale dataset containing many hours of video clips and accurate risk factors [6]. The video clips should also come from diverse real-

world game genres to ensure generalization. To our knowledge, such a comprehensive dataset does not exist.

In this paper, we present a new dataset called ‘VR.net’. It is publicly accessible via cloud storage² and possesses the following properties.

1. **Real-world, Diverse, and Large-scale:** VR.net comprises a collection of 165-hour gameplay videos featuring 100 real-world VR games from 10 representative genres. The dataset collection involves the participation of 500 individuals. Each game undergoes evaluation by at least ten different users, with each gameplay session lasting more than 10 minutes.
2. **Rich Labels:** Each video is recorded at a frame rate of 30 frames per second. For each frame, VR.net assigns 24 types of labels that are known to induce motion sickness. They are classified into two groups: ‘graphics’ and ‘interaction’. The ‘graphics’ labels describe game contents in each video frame, including camera location/speed/rotation, object location/speed/rotation, object semantic name, field of view, depth of field, and motion flow. The interaction labels describe a user’s reaction toward each frame, such as headset movement, joystick movement, gaze trajectory, and self-report. Fig 1 shows a snapshot of these labels.
3. **Accuracy:** VR.net aims to offer a clean dataset. We implement a data collection tool to automatically extract accurate ground-truth data (23 out of 24 features) from 3D game engines’ graphics pipelines. Only the self-reported Fast Motion Score (FSS) was collected manually. This approach enables us to obtain high-quality labeled data without accessing VR games’ source code. We offer public access to our data collection tool so that researchers can contribute to the expansion of VR.net.

*e-mail: elliott@ahlab.org

†e-mail: mark.billinghurst@auckland.ac.nz

‡e-mail: suranga@ahlab.org

¹<https://bloom.bg/3INVQ9O>

²<https://vrnet.ahlab.org>

Table 1: Comparison between VR.net and existing datasets

	VR.Net	Manual Labelled [7]	Vision-based [6]	Synthetic [5]
Dataset Size	165 Hours	<2 Hours	<2 Hours	< 5 Hours
Label Quality	Precise	Prone to Noise	Prone to Noise	Precise
Label Types	Rich (24)	Limited (3)	Limited (5)	Rich (11)
Diversity	Yes (100 real-world games from 10 distinct genres)	No (Passive videos)	No (Passive videos)	No (1 game)
Motion Sickness Measurement	FMS	SSQ	SSQ	SSQ

The rest of the paper is organized as follows. First, we summarize the differences between VR.net and other datasets in Section 2. We then demonstrate the construction process of VR.net and the implementation detail of our data collection platform in Section 3. Section 4 showcases several applications of the current VR.net, such as risk factor detection and comfort level prediction. We aim to show that VR.net can serve as a valuable resource for VR motion sickness research and broader machine-learning applications.

2 VR.NET AND RELATED DATASETS

In this section, we compare VR.Net with other related datasets. We summarize their differences in Table 1.

Manual Labeling: Several manual-labeled datasets have served as training and evaluation benchmarks for ML-based VR motion sickness research. For instance, VRSA [7] is acquired from the media-sharing platform Youtube. VRSA contains nine stereoscopic videos with beach, driving, and roller coaster scenes. Based on the scene movement speed, these videos are manually assigned three labels: ‘slow’, ‘normal’, and ‘intense’. VRSA has been utilized to demonstrate the correlation between exceptional movement patterns and VR sickness scores. However, the dataset contains less than one-hour video content and thus is insufficient to train a complex machine learning model. In contrast, VR.Net offers videos 100x the length of VRSA, allowing researchers to experiment with various state-of-the-art deep learning networks.

Vision-based Labeling: Several researchers applied computer vision models to automate video labeling and generate a larger-scale dataset. For instance, the study [6] assigned three labels (i.e., camera movement speed, direction, and depth) to 19 stereoscopic videos. These labels are approximated using principal component analysis on 2D pixel space. Similarly, Lee et al. [8] utilized the coarse-to-fine scale-invariant feature transform flow matching and the semi-global block matching to estimate motion flow and depth of field. Due to the heuristic nature of these vision algorithms, the generated label sets are now prone to data noise, eventually decreasing ML models’ performance. In contrast, VR.net retrieves accurate ground-truth data from a VR game’s graphics rendering pipeline. No noise is induced in this process.

Synthesized Datasets: Several datasets are gathered from purpose-built VR simulation environments (i.e., synthesized) [4, 5, 9, 10]. These environments usually provide users with passive VR experiences, such as a roller coaster or a space simulator. During a playthrough, the environments can capture comprehensive game scene parameters for automatic labeling. Though these synthesized datasets are accurate and more extensive in size, their data distribution might not reflect real-world VR games, which results in model generalization issues. In contrast, VR.net obtains labeled data from 100 off-the-shelf VR games in 10 representative genres, covering most real-world VR experiences.

Physiological Signal Datasets: Some datasets [11–13] aim to reveal the correlations between users’ physical responses and sickness levels. For instance, Kim et al. [14] curated a 4K 360-degree video dataset called VRPS DB-FR from Vimeo, aiming to investigate the interplay between framerate, physical response, and subjective scores. Lee et al. [15] employed a dataset named VRSA DB-Shaking, comprising twenty UHD 360-degree videos featuring diverse camera motions, along with the corresponding physiological signals recorded from individual viewers. However, these datasets provide limited content-related labels. Thus, they are less suitable for training a risk factor detector for VR content. Another notable distinction between VR.net and

existing datasets lies in their subjective score assessment methods. Existing datasets mostly adopt Kennedy Simulator Sickness Questionnaire (SSQ) [16], where users are asked to rate the amount of sickness they feel after each playthrough. SSQ consists of 16 physical symptoms related to VR sickness, each rated on a four-point scale for severity. On the other hand, VR.net adopts the Fast Motion Sickness Scale (FMS) [17], which allows users to provide almost real-time feedback on their comfort levels during the VR experience.

It should be noted that this paper draws inspiration from our prior research [18]. Distinctions arise in the following two aspects. Firstly, we have done a major upgrade to the tool described in the previous work. While the earlier tool exclusively captured low-level communication data between GPUs and CPUs, the data collection tool used in this paper is capable of directly capturing high-level graphics data from the 3D engine. Detailed enhancements are elaborated in Section 3.3. Secondly, the earlier paper primarily emphasizes the implementation of the data capture tool, whereas this paper centers around dataset construction. The previous paper presented a small demo dataset, whereas current paper contributes a real-world, large-scale, and diverse dataset directly applicable in motion sickness research.

3 CONSTRUCTING VR.NET

In this section, we describe our methodology for constructing VR.net.

3.1 Collecting Candidate VR Games

The first stage of VR.net construction is to select a vast number of real-world VR games from representative genres. To accomplish this target, we analyze content from the popular online game store Meta Quest³. Meta Quest places a strong emphasis on VR content, making it a superior choice over alternative game stores such as Steam⁴ or Epic⁵. As of May 2023, we manually compile a list of 1,410 VR games from the Meta store. We also gather relevant metadata, such as game descriptions and categories. It should be noted that the categories are assigned by the developers. They do not always correctly indicate the genre of games. Instead, they may describe a game’s selling price (e.g., free to play), visual properties (e.g., breathtaking scenery), or program features (e.g., cross-platform). To compensate for this discrepancy, we invited three independent researchers to perform content analysis coding on the data. The coders follow the validated Lucas and Sherry genre classification system [19]. They are allowed to assign multiple genres into a single game. Note that these coders are experienced gamers and have previously received training on the coding protocols. They are expected to categorize games more systematically and consistently. We also conducted a spot-check on 5% of the codes, and we did not discover any irregularities.

Finally, the following genres are included in our dataset: 1) Action, 2) Adventure, 3) Fighter, 4) Flight, 5) Music, 6) Puzzle, 7) Racing, 8) Shooter, 9) Simulation, 10) Sports. For each genre, we conducted random sampling to select 10 games (refer to the appendix for details).

3.2 Dataset Label Selection

The current version of VR.net offers approximately 165 hours of gameplay videos. For each x frame, VR.net automatically annotates a

³<https://www.meta.com/experiences/>

⁴<https://www.oculus.com/experiences/quest/>

⁵<https://store.epicgames.com/en-US/>

rich set of labels known to cause motion sickness. The labels are classified into ‘graphics’ and ‘interaction’. The graphics group describes the game contents in each video frame:

- **Camera Movement:** In computer graphics, each frame is generated from a camera’s perspective (in analogy to a real-life camera). Humans are more likely to experience nausea when the camera is moving than standing still [20–22]. Lo et al. [23, 24] have quantitatively investigated the effects of fast camera translational movement speed on VR sickness. VR sickness can also arise from excessive camera accelerating or decelerating movements [10, 25]. In addition, users can experience more significant discomfort when viewing a VR scene with rotational camera movements compared with translational movements [26]. VR.net records each camera’s ‘view’ matrix, which jointly encodes the camera location, velocity, acceleration, and rotation.
- **Field of View:** The field of view (FOV) is the extent of the observable game world at any given moment. Many studies [27, 28] have attributed the VR sickness to an overly wide FOV setting. Restricting the FOV can effectively relieve both subjective and objective symptoms of VR sickness [29]. VR.net captures each camera’s ‘projection’ matrix, which encodes FOV in a matrix form.
- **Depth of Field:** Depth of Field (DoF or depth texture) measures the distance from a viewing camera to every pixel on the screen. Carnegie et al. [30] reported that inappropriate DOF settings could cause visual discomfort.
- **Motion Vector:** Motion vectors capture the per-pixel, screen-space motion of objects from one frame to the next. Several studies [31, 32] have revealed that intensive motion flows are one of the major factors that cause simulator sickness.
- **Object Movement:** The movement pattern of each object can also be a crucial factor in inducing motion sickness. A typical explanation is ‘vection’, where stationary viewers feel like they have moved because nearby objects are approaching, receding, or rotating. Vection is believed to provoke motion sickness in susceptible individuals [25, 33]. VR.net captures the ‘model’ transformation matrix for each visible object, which jointly encodes an object’s location, velocity, acceleration, and rotation. VR.net also provides each object’s 3D bounding box, which can be used for object identification and collision detection.
- **Object Semantic Name:** Several studies [34, 35] indicate that different VR game contents may provoke varied levels and rates of nausea symptoms. In VR.net, we record each visible object’s name for semantic modeling of VR scenarios. The names are initially assigned by content developers to facilitate object management.

The interaction group describes how a user reacts to a video frame:

- **Headset Movement:** Headset movement is tightly associated with the motion sickness level [36, 37]. An excessive head movement may cause postural instability or vestibular-ocular reflex maladaptation, thus increasing the unpleasantness. VR.net retrieves the headset pose matrix from the underlying hardware, which accurately reflects a headset’s position, orientation, and spatial velocity.
- **Joystick Control:** Several studies [38] have shown that a user tends to report more severe VR sickness if the game restricts the user’s interactions with joysticks. Therefore, VR.net monitors each connected joystick’s pose matrix (i.e., position, orientation, and spatial velocity) and button actions.
- **Facial Expression:** Facial expressions have proven to be an objective physiological indicator of motion sickness level [39–41]. VR.net detects facial movements through VR headset sensors. These movements are converted into a set of 63 distinct expressions (e.g., jaw-dropping or nose wrinkling) aligning with the principles of the Facial Action Coding System (FACS) [42].

- **Gaze Tracking:** Inappropriate gaze positions are linked to increased levels of motion sickness [43–45]. VR.net incorporates built-in eye trackers within VR headsets to collect eye gaze information.
- **Physiological Signals:** Physiological signals (e.g., heart rate and heart rate variability) are effective indicators of motion sickness severity [46–48]. Most commercially available VR headsets do not possess built-in physiological sensors. VR.net gathers the signals when our participants have access to wearable heart rate sensors. In the current dataset release, physiological signals were collected from approximately 10% of participants. We anticipate a rise in this percentage in the future release.
- **Self Report:** VR.net adapts a widely-used measuring protocol called Fast Motion sickness Scale (FMS) [17] to collect momentary self-reported comfort scores from users. FMS asks the participant to verbally rate their experienced sickness every few minutes on a numerical scale (e.g., 1 to 5, where one indicates that the current scene does not incur motion sickness and five indicates that the scene results in severe discomfort). FMS is easy to administer and can collect measurements throughout a gameplay session. Verbal reports are also shown to be less distracting when compared to hand-action-based responses [49].

3.3 Data Collection Platform Implementation

VR Headsets: We select Meta Quest headsets as the data collection hardware for several compelling reasons. Firstly, Meta Quest headsets dominate the largest market share in the VR hardware industry [50]. As such, nearly every VR game is designed to be compatible with the headsets. Secondly, Meta Quest headsets possess the cutting edge of VR technology; they provide superior display quality and are equipped with advanced sensors (e.g., gaze and facial tracking). Lastly, Meta Quest features a user-friendly setup, eliminating the need for users to navigate complex hardware configurations.

Data Capturing Software: In our previous work [18], we purposed a data collection tool named VRHook to harness labeled data from real-world VR games without requiring access to their source code. Nevertheless, VRHook is not applicable for our purposes for several reasons. Firstly, VRHook captures very low-level GPU data, primarily comprising movement data of game objects. This data lacks more complex semantic labels, such as object names, bounding boxes, and motion vectors. Secondly, VRHook is tailored to the Windows platform and its graphics APIs, particularly DirectX. In contrast, Meta Quest headsets operate on the Android platform and employ distinct graphics APIs (e.g., OpenGL ES and Vulkan). Furthermore, Android enforces strict security policies and disallows process injection.

In this paper, we introduce a novel data collection tool called OculusHook. Our tool addresses the aforementioned challenges through two distinct approaches. Firstly, instead of capturing low-level GPU data, our tool intercepts high-level graphics rendering commands directly from 3D game engines. These rendering commands have a well-documented format, enabling precise extraction of more complex and semantic data. Second, our tool avoids the use of process injection. Instead, we employ static binary patching to incorporate our label recording logic into a VR game’s binary. This method demands no access to the game’s source code and does not violate Android’s security policies. We provide a web application⁶ to automate this patching process. Researchers can upload their game copy to generate a patched binary, facilitating them to contribute to the expansion of VR.net.

Figure 2 demonstrates the main workflow of our data collection tool. Given a VR game installation package (i.e., APK), our tool first extracts its engine files (e.g., libUnity.so or libUnreal.so). Then our tool inserts trampolines [51] to core engine functions to facilitate data logging. When the game invokes the engine function ‘GetCamera’, the inserted trampoline will first redirect the execution flow to our logging logic to record the camera-related data. The original function execution

⁶<https://vrpatcher.ahlab.org>

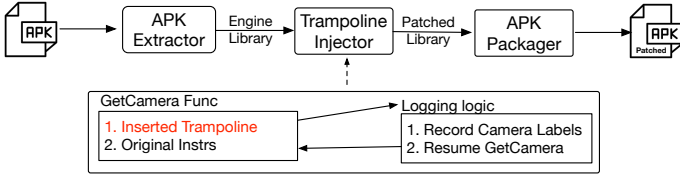


Fig. 2: The workflow of our data collection tool.

will be resumed subsequently. Finally, our tool repackages the library files, creating a patched game package ready for data collection. Our tool currently supports two widely-used 3D game engines: Unity and Unreal. The two engines are the backbone of the modern game industry and power almost every VR game. Out of the 1,410 games from the Meta Quest store, a significant portion, 1,023 (72%), are developed using the Unity engine, while 380 (26%) are built upon the Unreal engine. We present implementation details for extracting dataset labels below.

1. **Camera Movement and FOV:** For Unity, we can intercept an engine interface named `Camera.getAllCameras()`, which returns an array of camera object references in the current scene. We then can access each object’s `worldToCameraMatrix` and `projectMatrix` properties to obtain its view matrix and projection matrix. They encode camera movement and FOV, receptively.

For Unreal, we first locate a global symbol named ‘GObject’, which maintains an array of metadata for all game objects. We then can filter the objects with a class tag ‘ACameraActor’ to locate all cameras. Next, we can intercept a member function called ‘Engine.CameraActor.GetCameraComponent’ to retrieve a ‘ViewInfo’ object, which encapsulates the camera’s view and projection matrix.

2. **Object Movement and Semantic Name:** For Unity, we intercept an API named `object.FindObjectOfType<Renderer>` to obtain a list of active `renderer` instances in the current scene. A `renderer` instance corresponds to a 3D object visible on the screen. We can visit an instance’s ‘`localToWorldMatrix`’ property to access its model transformation matrix. We can also access its ‘`name`’ and ‘`bounds`’ members to capture the object’s semantic name and 3D bounding box.

Regarding Unreal, we first filter out objects with the class tag ‘AActor’ from the ‘GObject’ array. These objects correspond to visible 3D objects in the game. We can then extract their model transformation matrices and semantic names by invoking the member functions ‘`GetTransform`’ and ‘`GetActorNameOrLabel`’.

3. **DOF and Motion Vector:** For Unity, we can intercept the ‘`Camera.Main`’ function to get an object reference of the active camera. We then update the camera’s `depthTextureMode` property to the combination of two flags `DepthTextureMode.Depth` and `DepthTextureMode.MotionVectors`. This instructs Unity to generate depth and motion vector textures for each frame internally. To access the texture contents, we can invoke an interface named `Shader.GetGlobalTexture` with parameters ‘`CameraDepthTexture`’ and ‘`CameraMotionVectorsTexture`’. They reveal native GPU resource pointers, which we then can copy to the system memory via the API ‘`Graphics.copyTexture`’.

Unreal internally computes depth and motion vector texture to implement motion blur and temporal anti-aliasing effects. We can intercept a member function named ‘`FDeferredShaderingSceneRender::RenderVelocities`’, which takes the native pointers to depth texture and motion vector texture as parameters.

4. **Interaction Labels:** We can intercept Meta Quest’s VR runtime interfaces to capture the interaction labels, for instance, `ovrp_GetControllerState` for joystick movement, `ovrp_GetEyeGazesState` for gaze tracking, and

`ovrp_GetFaceState` for facial expressions. To implement the verbal response self-report method, our tool conducts the voice recording via Android’s Microphone API. We adopt the speech recognition toolkit *Vosk*⁷ to recognize users’ voice responses automatically. We utilize the largest voice model *vosk-model-en-us-0.42-gigaspeech* to achieve high-accuracy recognition.

3.4 Data Collection Procedure

Ethics Approval and Human Participant Data Handling: We obtained ethical approval from the Ethics Committee at the University of Auckland. All participants were provided with a participant information sheet and a consent form, ensuring that they had a comprehensive understanding of the study before data collection commenced. Furthermore, the participants retain the right to withdraw from the study at any point if they feel overwhelmed or for any other reason. All personally identifiable information collected during the study was anonymized prior to releasing our dataset. This involves carefully removing or obfuscating any identifiers that could potentially link data back to individual participants. Researchers granted access to our dataset are obligated to use it exclusively for the intended research purposes.

Participants: Through email and posters, we recruited 500 participants. They are students, faculty, and staff from three universities across three continents.

Procedure: The data was collected in a lab setting. We provide all participants with a Meta Quest Pro VR headset. We patched and installed all selected VR games into the headset beforehand. Each participant was randomly assigned games from different genres. This assignment was done in a way that each game is played by at least five different users. We instructed users to play every assigned game for around 15 minutes. During the game sessions, we follow the FMS protocol and ask users to report their momentary self-reported scores verbally. To encourage self-reporting, we give a verbal reminder to users every 30 seconds. Participants could also voluntarily report whenever they felt a change in their discomfort level.

Dataset Export and Summary: We implement a Python script to process users’ data files automatically. The output data is then exported in a CSV format, where each row contains a single video frame and the corresponding labels. We also follow Google’s Data Cards Playbook framework and provide structured reports about our dataset in the supplementary document.

4 UTILIZATION OF VR.NET

In this section, we showcase two essential utilities of VR.net. Firstly, VR.net can serve as a data set to construct risk factor detection systems for VR content. We provide an example in section 4.1 to detect whether the camera is performing an excessive movement (i.e., fast speed, violent acceleration, and multi-axis rotation). Secondly, VR.net can be used as a benchmark dataset to reproduce and validate results from previous motion sickness studies. In section 4.3, we replicate a previous study that exploits the motion flow and depth texture labels to predict motion sickness levels. These examples underline the advantages of having a large-scale dataset with diverse VR games and rich labels.

4.1 Excessive Camera Movement Detection

We build a proof-of-concept system to predict camera-related risk factors, as shown in Fig. 3. The model consumes a 1-second video stream and outputs three binary numbers, indicating the presence of the following risk factors:

1. Fast camera speed: Whether the camera speed is faster than 3 m/s, at which most users will start experiencing VR sickness [24].
2. Excessive camera acceleration: Whether the camera acceleration is above a threshold of 0.7 m/s², at which participants start to feel temporary symptoms of cybersickness [52].
3. Multi-axis rotation: Whether the camera executes a rotation movement involving more than one axis. Such rotation can significantly

⁷<https://alphacephei.com/vosk/>

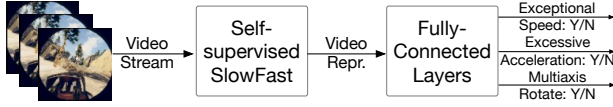


Fig. 3: A pipeline to detect motion sickness risk factors.

Table 2: Pipeline performance. Gray and Cray colors correspond to track-based splitting and user-based splitting, respectively.

Risk Factors	Accuracy	F1 Score
Excessive Speed	0.76 / 0.82	0.71 / 0.74
Excessive Acceleration	0.75 / 0.79	0.73 / 0.75
Multi-axis Rotation	0.71 / 0.74	0.73 / 0.77

escalate the severity of discomfort [53]. More specifically, given two consecutive view matrices V_1 and V_2 , we can calculate the camera rotation difference via the formula: $V_\Delta = V_2 * V_1^{-1}$, where V_Δ can be further decomposed into three elementary Euler angles, commonly referred to yaw, roll, and pitch. If two of them exceed a predefined value [26], we detect a multi-axis rotation.

Our model is built upon *SlowFast* [54], a commonly-used self-supervised learning model for video recognition. Self-supervised learning is a process where the model first trains itself to learn one part of the input from another part of the input. For example, given the upper half of the video frame, the model can learn how to predict the lower half of the frame. By doing this, the model can obtain useful representations of input videos, which can be later fine-tuned with a limited number of labels for actual supervised tasks. Self-supervised learning generally outperforms conventional supervised learning, particularly when the training dataset is small [55–57].

Since *SlowFast* already provides a model checkpoint pre-trained on sizeable video datasets, we could directly attach the model to fully connected layers to output the risk factor labels. Straightforwardly, we could fine-tune three fully connected layers, each of which outputs an individual label. The merit is that it allows us to add more risk factors in the future without retraining the existing networks. Nevertheless, we opt for an alternative architecture, where we fine-tune a single fully connected layer and output three labels together. This design exploits the fact that all the tasks may share the same underlying information. Training them together in one model can achieve higher performance (i.e., multi-task learning [58]).

We utilize the PyTorch [59] framework to implement the *SlowFast* model. We select a roller coaster game in VR.net for model training. The game provides five roller coaster tracks on different terrains (e.g., Snow Mountain and Urban City). Each track lasts approximately 6 minutes and is evaluated by ten different users. This generates a dataset with 5-hour video clips and the corresponding risk factor labels. We first split the dataset into training and test sets in a ratio of 80/20. In our implementation, two splitting mechanisms are evaluated: 1) four tracks for training and one track for testing, and 2) four users for training and one user for testing. We set the number of epochs to 5 and the batch size to 32. The learning rate is $1e^{-4}$. We use two V100 32 GB GPUs, and the training wall time is 12 hours. The performance of our system for both data-splitting mechanisms is shown in Table 2. Our system can predict three risk factors with reasonable accuracy regardless of the data-splitting mechanism. This also suggests that our model offers decent generalization performance across game content and users.

4.2 Object Density Estimation

Previous studies [34] observed that a higher scene complexity is more likely to induce motion sickness. For example, flight simulation gamers tend to perceive more severe sickness in a low-altitude flight than in a high-altitude one because the former scenario has more visual stimuli, such as buildings, trees, and cars. Here, we exploit VR.net to build an object density estimator given a one-second gameplay video.

Specifically, we fine-tuned the state-of-the-art video recognition model VideoMAE [60] with the music game ‘Beat Saber’ data in VR.net. The game was played by five users and each session lasted on average 3 minutes (i.e., 900 seconds in total). We pre-processed the object movement data to exclude objects whose bounding boxes are too small. Those excluded objects are usually helper objects (e.g., light waves), which do not appear on the rendered frames. We then counted the distinct objects in one second and discretized the object numbers to generate object density classes: low (0-100 objects), medium (100-150 objects), and high (>150 objects). These thresholds are chosen to provide an approximately uniform data distribution across these three classes. Two sets of video data were used; In the first set, the videos were encoded at a frame rate of 30 FPS as usual. In the second set, the videos are resampled with 0.5X speed to increase the dataset size by a factor of 2. These videos are then divided into the train, validation, and test sets in the proportion of 80/10/10. The 3-class classification accuracy of these models is provided in Table 3. This result shows that our simple way of exploiting the object labels can provide high performance for the object density estimation task. Our future investigations will include extending the models for more games.

Table 3: Object density estimation performance.

FPS	Train/Val/Test	Accuracy
30	723 / 90 / 90	0.94
15	1452 / 180 / 180	0.97

4.3 Comfort Rating Prediction

In several previous studies [6, 8], a convolutional neural network (CNN) has been deployed to predict the degree of motion sickness, as shown in Figure 4. The input features of the network include a sequence of motion flow frames and a sequence of depth texture frames. The output is a comfort rating on a scale from 1 to 5 (a higher number corresponds to a higher level of discomfort). The CNN architecture is three-dimensional, which can perform convolution operations in the two-spatial dimensions (i.e., horizontal and vertical directions of a frame) and the temporal dimension (i.e., several consecutive frames). This enables expressing the degree of motion sickness with multiple frames as a single value. Despite these achievements, these studies unanimously pointed out that the size of their training dataset limits their predictor performance. Here, we can use VR.net as a benchmark dataset to replicate and validate this motion sickness research.

We reuse the roller coaster game data in the first application. The video clips are encoded at a resolution of 1832×960 . We downscale each frame’s size by 8 (i.e., 229×120) to reduce computational cost. Amid the video stream, we discover 471 self-reported scores. We assign each score to a 3-second gameplay video clip right before the report. This generates a sub-dataset that contains 471 three-second video clips with their comfort ratings. The clip length is motivated by previous research [61], which suggests that 3 seconds is sufficient for users to interpret VR content and generate a momentary cybersickness response. A deep-learning model typically needs a considerable amount of training data. We thus use a simple-yet-effective pixel-shifting technique to augment our dataset. Specifically, we shift the motion flow and depth texture by -5 and +5 pixels on horizontal and vertical axes with zero padding. Therefore, the amount of data increased to 4 times the original amount. Since it is challenging for human eyes to recognize the differences between pixel-shifted frames, we can assume that the motion sickness induced by the shifted dataset is the same as that original dataset.

We first split our dataset into training and test sets in a ratio of 90/10. We then utilized the PyTorch framework to implement and train the 3D CNN architecture. All experiments were performed on the same server used in the first application. We set the learning rate to $1e^{-4}$, and the total training time is approximately 14 hours. Our results show that the model provides a decent prediction accuracy of 0.70. Guided by the existing studies [6, 8], we test the significance of the Pearson correlation coefficient between prediction and ground truth. We observed that the

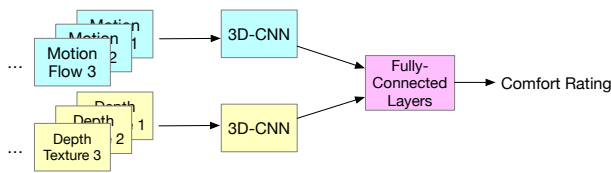


Fig. 4: A 3D-CNN pipeline to predict sickness scores.

Pearson coefficient is strongly positive (0.74) with a p-value less than 0.05. This validates the previous finding that motion flow and depth of field play an essential role in sickness level prediction.

5 DISCUSSION AND FUTURE WORK

Extending the size of the dataset: We are continuing to expand the VR.net dataset. To speed up the construction process, we plan to explore crowdsourcing marketplaces (e.g., Amazon Mechanical Turk) to recruit more participants. We also aim to deliver VR.net to wider research communities by making it more easily accessible. For instance, we are experimenting with InterPlanetary File System⁸ to enable the high-speed distribution of sizable video data in VR.net. In addition, we attempt to foster a VR.net community by developing an online platform where everyone can contribute to and benefit from VR.net resources.

Building a full-fledged risk factor detection pipeline: We are exploring the potential of rich labels in VR.net. Our primary goal is to increase the number of detectable risk factors in the pipeline in Fig. 3. For instance, we can identify overly wide FOV, at which users are more susceptible to motion sickness [62]. We can also detect the high magnitude of motion flow, which means the frame contains complex visual information and may increase the extent of the sickness [31]. We envision the full-fledged pipeline can be integrated into 3D design software such as Unreal and Unity editor. It can help content developers identify and mitigate risk factors in real-time.

Limitations: VR games that do not build upon Unity and Unreal engines are excluded from our dataset. This is acceptable at this stage as they are scarce and do not significantly impact the large-scale and diverse properties of VR.net. In addition, our dataset does not include some less-known risk factors labels such as audio effects [63] and graphic realism [64]. More research effort on them is warranted. The current version of our dataset utilizes the FMS method to capture participants' momentary motion sickness scores during gameplay. In the future release, we aim to integrate additional motion sickness assessment methods like 16-item SSQ.

6 CONCLUSION

This paper presents 'VR.net', a large-scale video dataset that aims to stimulate new ML-based VR sickness research and beyond. VR.net is sizable and covers a diverse range of real-world VR experiences. In addition, VR.net provides rich label sets that are known to induce motion sickness. These ground truth labels are automatically and precisely extracted from 3D engines' rendering pipelines. We showcase how VR.net can be used to build a machine-learning pipeline for risk factor detection and comfort rating estimation. VR.net is constantly expanding. We envision VR.net can serve as a valuable resource for VR motion sickness research and broader machine-learning applications.

REFERENCES

- [1] D. M. Johnson. Introduction to and review of simulator sickness research. 2005. 1
- [2] D. Saredakis, A. Szpak, B. Birchead, H. A. Keage, A. Rizzo, and T. Loetscher. Factors associated with virtual reality sickness in head-mounted displays: a systematic review and meta-analysis. *Frontiers in human neuroscience*, 14:96, 2020. 1
- [3] E. Chang, H. T. Kim, and B. Yoo. Virtual reality sickness: a review of causes and measurements. *International Journal of Human-Computer Interaction*, 36(17):1658–1682, 2020. 1
- [4] H. Oh and W. Son. Cybersickness and its severity arising from virtual reality content: A comprehensive study. *Sensors*, 22(4):1314, 2022. 1, 2
- [5] S. Hell and V. Argyriou. Machine learning architectures to predict motion sickness using a virtual reality rollercoaster simulation tool. In *2018 IEEE International Conference on Artificial Intelligence and Virtual Reality (AIVR)*, pp. 153–156. IEEE, 2018. 1, 2
- [6] N. Padmanaban, T. Ruban, V. Sitzmann, A. M. Norcia, and G. Wetzstein. Towards a machine-learning approach for sickness prediction in 360 stereoscopic videos. *IEEE transactions on visualization and computer graphics*, 24(4):1594–1603, 2018. 1, 2, 5
- [7] H. G. Kim, H.-T. Lim, S. Lee, and Y. M. Ro. Vrsa net: Vr sickness assessment considering exceptional motion for 360 vr video. *IEEE transactions on image processing*, 28(4):1646–1660, 2018. 2
- [8] T. M. Lee, J.-C. Yoon, and I.-K. Lee. Motion sickness prediction in stereoscopic videos using 3d convolutional neural networks. *IEEE transactions on visualization and computer graphics*, 25(5):1919–1927, 2019. 2, 5
- [9] S. Martirosov, M. Bureš, and T. Zítka. Cyber sickness in low-immersive, semi-immersive, and fully immersive virtual reality. *Virtual Reality*, 26(1):15–32, 2022. 2
- [10] J. Kim, W. Kim, S. Ahn, J. Kim, and S. Lee. Virtual reality sickness predictor: Analysis of visual-vestibular conflict and vr contents. In *2018 tenth international conference on quality of multimedia experience (QoMEX)*, pp. 1–6. IEEE, 2018. 2, 3
- [11] C.-Y. Liao, S.-K. Tai, R.-C. Chen, and H. Hendry. Using eeg and deep learning to predict motion sickness under wearing a virtual reality device. *Ieee Access*, 8:126784–126796, 2020. 2
- [12] J. Smyth, S. Birrell, R. Woodman, and P. Jennings. Exploring the utility of eda and skin temperature as individual physiological correlates of motion sickness. *Applied Ergonomics*, 92:103315, 2021. 2
- [13] T. Xue, A. El Ali, T. Zhang, G. Ding, and P. Cesar. Ceap-360vr: A continuous physiological and behavioral emotion annotation dataset for 360 vr videos. *IEEE Transactions on Multimedia*, 2021. 2
- [14] H. G. Kim, S. Lee, S. Kim, H.-t. Lim, and Y. M. Ro. Towards a better understanding of vr sickness: physical symptom prediction for vr contents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 836–844, 2021. 2
- [15] S. Lee, J. U. Kim, H. G. Kim, S. Kim, and Y. M. Ro. Saca net: cyber-sickness assessment of individual viewers for vr content via graph-based symptom relation embedding. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXIII 16*, pp. 170–186. Springer, 2020. 2
- [16] R. S. Kennedy, N. E. Lane, K. S. Berbaum, and M. G. Lilienthal. Simulator sickness questionnaire: An enhanced method for quantifying simulator sickness. *The international journal of aviation psychology*, 3(3):203–220, 1993. 2
- [17] B. Keshavarz and H. Hecht. Validating an efficient method to quantify motion sickness. *Human factors*, 53(4):415–426, 2011. 2, 3
- [18] E. Wen, T. I. Kaluarachchi, S. Siriwardhana, V. Tang, M. Billinghurst, R. W. Lindeman, R. Yao, J. Lin, and S. Nanayakkara. Vrhook: A data collection tool for vr motion sickness research. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–9, 2022. 2, 3
- [19] M. Foxman, D. Beyea, A. P. Leith, R. A. Ratan, V. H. H. Chen, and B. Klebig. Beyond genre: Classifying virtual reality experiences. *IEEE Transactions on Games*, 14(3):466–477, 2021. 2
- [20] F. Bonato, A. Bubka, S. Palmisano, D. Phillip, and G. Moreno. Vection change exacerbates simulator sickness in virtual environments. *Presence: Teleoperators and Virtual Environments*, 17(3):283–292, 2008. 3
- [21] J.-R. Chardonnet, M. A. Mirzaei, and F. Mérenne. Visually induced motion sickness estimation and prediction in virtual reality using frequency components analysis of postural sway signal. In *International conference on artificial reality and telexistence eurographics symposium on virtual environments*, pp. 9–16, 2015. 3
- [22] C.-L. Liu and S.-T. Uang. A study of sickness induced within a 3d virtual store and combated with fuzzy control in the elderly. In *2012 9th International Conference on Fuzzy Systems and Knowledge Discovery*, pp. 334–338. IEEE, 2012. 3
- [23] R. H. So, W. Lo, and A. T. Ho. Effects of navigation speed on motion sickness caused by an immersive virtual environment. *Human factors*, 43(3):452–461, 2001. 3
- [24] R. H. So, A. Ho, and W. Lo. A metric to quantify virtual scene movement for the study of cybersickness: Definition, implementation, and verification. *Presence*, 10(2):193–215, 2001. 3, 4

⁸<https://ipfs.tech/>

- [25] B. Keshavarz, B. E. Riecke, L. J. Hettinger, and J. L. Campos. Vection and visually induced motion sickness: how are they related? *Frontiers in psychology*, 6:472, 2015. 3
- [26] F. Bonato, A. Bubka, and S. Palmisano. Combined pitch and roll and cybersickness in a virtual environment. *Aviation, space, and environmental medicine*, 80(11):941–945, 2009. 3, 5
- [27] H.-L. Duh, J. Lin, R. V. Kenyon, D. E. Parker, and T. A. Furness. Effects of field of view on balance in an immersive environment. In *Proceedings IEEE Virtual Reality 2001*, pp. 235–240. IEEE, 2001. 3
- [28] J.-W. Lin, H. B.-L. Duh, D. E. Parker, H. Abi-Rached, and T. A. Furness. Effects of field of view on presence, enjoyment, memory, and simulator sickness in a virtual environment. In *Proceedings IEEE virtual reality 2002*, pp. 164–171. IEEE, 2002. 3
- [29] I. B. Adhanom, N. N. Griffin, P. MacNeilage, and E. Folmer. The effect of a foveated field-of-view restrictor on vr sickness. In *2020 IEEE conference on virtual reality and 3D user interfaces (VR)*, pp. 645–652. IEEE, 2020. 3
- [30] K. Carnegie and T. Rhee. Reducing visual discomfort with hmds using dynamic depth of field. *IEEE computer graphics and applications*, 35(5):34–41, 2015. 3
- [31] J.-Y. Lee, P.-H. Han, L. Tsai, R.-D. Peng, Y.-S. Chen, K.-W. Chen, and Y.-P. Hung. Estimating the simulator sickness in immersive virtual reality with optical flow analysis. In *SIGGRAPH Asia 2017 Posters*, pp. 1–2. 2017. 3, 6
- [32] S. H. Park, B. Han, and G. J. Kim. Mixing in reverse optical flow to mitigate vection and simulation sickness in virtual reality. In *CHI Conference on Human Factors in Computing Systems*, pp. 1–11, 2022. 3
- [33] B. Keshavarz, A. E. Philipp-Muller, W. Hemmerich, B. E. Riecke, and J. L. Campos. The effect of visual motion stimulus characteristics on vection and visually induced motion sickness. *Displays*, 58:71–81, 2019. 3
- [34] J. Kim, H. Oh, W. Kim, S. Choi, W. Son, and S. Lee. A deep motion sickness predictor induced by visual stimuli in virtual reality. *IEEE Transactions on Neural Networks and Learning Systems*, 2020. 3, 5
- [35] K. Nesbitt, S. Davis, K. Blackmore, and E. Nalivaiko. Correlating reaction time and nausea measures with traditional measures of cybersickness. *Displays*, 48:1–8, 2017. 3
- [36] J. Li, A. Reda, and A. Butz. Queasy rider: How head movements influence motion sickness in passenger use of head-mounted displays. In *13th International Conference on Automotive User Interfaces and Interactive Vehicular Applications*, pp. 28–38, 2021. 3
- [37] L. Tychsen and P. Foeller. Effects of immersive virtual reality headset viewing on young children: visuomotor function, postural stability, and motion sickness. *American journal of ophthalmology*, 209:151–159, 2020. 3
- [38] X. Dong, K. Yoshida, and T. A. Stoffregen. Control of a virtual vehicle influences postural activity and motion sickness. *Journal of Experimental Psychology: Applied*, 17(2):128, 2011. 3
- [39] M. Arima, K. Ikeda, and R. Hosoda. Analyses of facial expressions for the evaluation of seasickness. In *Oceans' 04 MTS/IEEE Techno-Ocean '04 (IEEE Cat. No. 04CH37600)*, vol. 2, pp. 1129–1132. IEEE, 2004. 3
- [40] R. Tan, R. Gao, W. Li, K. Cao, Y. Li, C. Lv, F.-Y. Wang, and D. Cao. Xsickness in intelligent mobile spaces and metaverses. *IEEE Intelligent Systems*, 37(5):86–94, 2022. 3
- [41] M. Beggiano, N. Rauh, and J. Krems. Facial expressions as indicator for discomfort in automated driving. In *Intelligent Human Systems Integration 2020: Proceedings of the 3rd International Conference on Intelligent Human Systems Integration (IHSI 2020): Integrating People and Intelligent Systems, February 19-21, 2020, Modena, Italy*, pp. 932–937. Springer, 2020. 3
- [42] P. Ekman and W. V. Friesen. Facial action coding system. *Environmental Psychology & Nonverbal Behavior*, 1978. 3
- [43] S. Wibirama, H. A. Nugroho, and K. Hamamoto. Depth gaze and ecg based frequency dynamics during motion sickness in stereoscopic 3d movie. *Entertainment computing*, 26:117–127, 2018. 3
- [44] C. Diels, K. Ukai, and P. A. Howarth. Visually induced motion sickness with radial displays: effects of gaze angle and fixation. *Aviation, space, and environmental medicine*, 78(7):659–665, 2007. 3
- [45] S. Wibirama and K. Hamamoto. Investigation of visually induced motion sickness in dynamic 3d contents based on subjective judgment, heart rate variability, and depth gaze behavior. In *2014 36th annual international conference of the IEEE engineering in medicine and biology society*, pp. 4803–4806. IEEE, 2014. 3
- [46] S. R. Holmes and M. J. Griffin. Correlation between heart rate and the severity of motion sickness caused by optokinetic stimulation. *Journal of Psychophysiology*, 15(1):35, 2001. 3
- [47] K. Zuzewicz, A. Saulewicz, M. Konarska, and Z. Kaczorowski. Heart rate variability and motion sickness during forklift simulator driving. *International Journal of Occupational Safety and Ergonomics*, 17(4):403–410, 2011. 3
- [48] L. T. LaCount, V. Napadow, B. Kuo, K. Park, J. Kim, E. N. Brown, and R. Barbieri. Dynamic cardiovagal response to motion sickness: a point-process heart rate variability study. In *2009 36th Annual Computers in Cardiology Conference (CinC)*, pp. 49–52. IEEE, 2009. 3
- [49] D. R. Proffitt, M. Bhalla, R. Gossweiler, and J. Midgett. Perceiving geographical slant. *Psychonomic bulletin & review*, 2:409–428, 1995. 3
- [50] Meta captures 90% of vr headset market share. <https://www.insiderintelligence.com/content/meta-captures-90-of-vr-headset-market-share>. 3
- [51] N. Totosis and C. Patsakis. Android hooking revisited. In *2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech)*, pp. 552–559. IEEE, 2018. 3
- [52] L. Terenzi and P. Zaal. Rotational and translational velocity and acceleration thresholds for the onset of cybersickness in virtual reality. In *AIAA Scitech 2020 Forum*, p. 0171, 2020. 4
- [53] B. Keshavarz and H. Hecht. Axis rotation and visually induced motion sickness: the role of combined roll, pitch, and yaw motion. *Aviation, space, and environmental medicine*, 82(11):1023–1029, 2011. 5
- [54] C. Feichtenhofer, H. Fan, J. Malik, and K. He. Slowfast networks for video recognition. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 6202–6211, 2019. 5
- [55] S. Naem, A. Ali, S. Anam, and M. M. Ahmed. An unsupervised machine learning algorithms: Comprehensive review. *Int. J. Comput. Digit. Syst*, 2023. 5
- [56] M. Alloghani, D. Al-Jumeily, J. Mustafina, A. Hussain, and A. J. Aljaaf. A systematic review on supervised and unsupervised machine learning algorithms for data science. *Supervised and unsupervised learning for data science*, pp. 3–21, 2020. 5
- [57] S. Solorio-Fernández, J. A. Carrasco-Ochoa, and J. F. Martínez-Trinidad. A review of unsupervised feature selection methods. *Artificial Intelligence Review*, 53(2):907–948, 2020. 5
- [58] C. Fifty, E. Amid, Z. Zhao, T. Yu, R. Anil, and C. Finn. Efficiently identifying task groupings for multi-task learning. *Advances in Neural Information Processing Systems*, 34:27503–27516, 2021. 5
- [59] H. Fan, T. Murrell, H. Wang, K. V. Alwala, Y. Li, Y. Li, B. Xiong, N. Ravi, M. Li, H. Yang, et al. Pytorchvideo: A deep learning library for video understanding. In *Proceedings of the 29th ACM International Conference on Multimedia*, pp. 3783–3786, 2021. 5
- [60] Z. Tong, Y. Song, J. Wang, and L. Wang. Videomae: Masked autoencoders are data-efficient learners for self-supervised video pre-training. *arXiv preprint arXiv:2203.12602*, 2022. 5
- [61] N. McHugh, S. Jung, S. Hoermann, and R. W. Lindeman. Investigating a physical dial as a measurement tool for cybersickness in virtual reality. In *25th ACM Symposium on Virtual Reality Software and Technology*, pp. 1–5, 2019. 5
- [62] M. Emoto, M. Sugawara, and Y. Nojiri. Viewing angle dependency of visually-induced motion sickness in viewing wide-field images by subjective and autonomic nervous indices. *Displays*, 29(2):90–99, 2008. 6
- [63] O. X. Kuiper, J. E. Bos, C. Diels, and E. A. Schmidt. Knowing what's coming: Anticipatory audio cues can mitigate motion sickness. *Applied Ergonomics*, 85:103068, 2020. 6
- [64] J. F. Golding, K. Doolan, A. Acharya, M. Tribak, and M. A. Gresty. Cognitive cues and visually induced motion sickness. *Aviation, space, and environmental medicine*, 83(5):477–482, 2012. 6